Enterprise Knowledge Intelligence Platform

A Project Based Learning Report submitted in partial fulfilment of the requirements

for the award of the degree

Bachelor of Technology

Interdisciplinary Project in

Database Systems, Algorithms, Operating Systems and Machine Learning

Submitted by
SathyaMaragani



Koneru Lakshmaiah Education Foundation

Aziz Nagar

Aziz Nagar - 500075

October 2026

Abstract

The Enterprise Knowledge Intelligence Platform is an integrated system for uploading, organizing, protecting, searching, analyzing and retrieving organizational knowledge. It combines a React user interface with a Spring Boot service and assigns distinct storage responsibilities to PostgreSQL, MongoDB and Qdrant. PostgreSQL is the relational source of truth for identities, roles, permissions, document metadata and search history; MongoDB stores extracted text, processing metadata and chunks; Qdrant stores 384-dimensional MiniLM vectors for semantic retrieval. The platform supports keyword, fuzzy, semantic and hybrid search, role-based access control, document sharing, administration, activity history and a document viewer. Its upload pipeline accepts PDF, DOCX, TXT and Markdown files, extracts text securely, chunks it into overlapping windows and indexes it for retrieval. The TextHack engine contributes 20 algorithms implemented from first principles, while the machine-learning work evaluates retrieval, category classification and clustering with leakage-resistant protocols. The OSSP contribution, ShellForge, demonstrates process creation, signal handling, pipes, redirection and memory-safe systems programming. The deployed application runs on Vercel and Render with Neon, MongoDB Atlas and Qdrant Cloud. Verification includes 191 backend tests, 148 frontend tests, 452 TextHack assertions, 148 ShellForge assertions, machine-learning checks and 28 deployment smoke checks. The result is a working, deployed academic platform that demonstrates how databases, algorithms, operating systems and machine learning can be combined into one coherent knowledge-retrieval product.

List of Figures

Figure 1. Logical architecture of the integrated platform 6
Figure 2. Document ingestion, chunking and indexing workflow 7
Figure 3. Unified search modes, permission filtering and ranking 8
Figure 4. Cloud deployment topology 13

List of Tables

Table 1. Interdisciplinary workstreams and responsibilities 5
Table 2. Technology stack 6
Table 3. Storage responsibilities 6
Table 4. Search modes 8
Table 5. Representative REST endpoints 9
Table 6. Implemented TextHack algorithms 9
Table 7. Retrieval evaluation on the bounded FiQA subset 10
Table 8. Verification summary 11
Table 9. Selected project outcomes 12
Table 10. Production services 13

Table of Contents

1. INTRODUCTION 5
1.1 Background and Motivation 5
1.2 Problem Statement 5
1.3 Project Objectives 5
1.4 Project Scope and Workstreams 5
2. SYSTEM DESIGN AND METHODOLOGY 6
2.1 Overall Architecture 6
2.2 Hybrid Database Responsibility Model 6
2.3 Document Ingestion Method 7
2.4 Search Methodology 8
2.5 Security and Access Model 8
3. IMPLEMENTATION 8
3.1 Backend and API 8
3.2 Frontend Experience 9
3.3 TextHack Algorithm Engine 9
3.4 Machine Learning Pipeline 10
3.5 Classification, Clustering and Insights 10
3.6 ShellForge Systems Programming 10
4. EXPERIMENTS AND VALIDATION 11
4.1 Automated Test Strategy 11
4.2 PDF and DOCX Validation 11
4.3 Retrieval and Model Evaluation 11
4.4 Resource and Deployment Testing 11
5. RESULTS AND DISCUSSION 12
5.1 Completed Capabilities 12
5.2 Quantitative Outcomes 12
5.3 Limitations 12
6. DEPLOYMENT AND OPERATIONS 13
6.1 Cloud Topology 13
6.2 Production Safeguards 13
6.3 Live Availability 13
7. CONCLUSION AND FUTURE WORK 14
7.1 Conclusion 14
7.2 Future Work 14
REFERENCES 15

Enterprise Knowledge Intelligence Platform

1. INTRODUCTION

1.1 Background and Motivation

Organizations accumulate policies, procedures, research notes, technical guides and operational records across many repositories. Conventional file storage preserves the documents but does not provide a unified way to understand their content, enforce document-level access or find material by meaning. This project addresses that gap with an enterprise knowledge platform that combines structured metadata, full document text, vector embeddings and algorithmic text analysis behind one authenticated web interface.

The project is deliberately interdisciplinary. Database engineering provides consistent identity and metadata management; data structures and algorithms provide transparent search and analysis primitives; operating-systems work demonstrates the low-level process and IPC foundations used by software platforms; and machine learning supplies measured semantic retrieval, classification and clustering. The completed repository therefore serves both as a usable application and as evidence for four subject areas.

1.2 Problem Statement

The central problem is to let authorized users store heterogeneous documents and retrieve the right information without forcing every question into exact keywords. The system must reconcile three kinds of data: relational business metadata and permissions, unstructured document content, and numerical embedding vectors. It must also remain testable, explainable and deployable on limited infrastructure.

1.3 Project Objectives

- Provide secure authentication, role-based authorization and document-level sharing.
- Accept PDF, DOCX, plain-text and Markdown documents and convert their text into a common searchable representation.
- Implement keyword, fuzzy, semantic and hybrid search with consistent permission filtering.
- Store relational metadata, document content and vector representations in databases suited to each responsibility.
- Implement and demonstrate core algorithms without replacing them with library shortcuts.
- Evaluate retrieval, classification and clustering using reproducible datasets and leakage-resistant protocols.
- Build a Unix-like teaching shell that demonstrates processes, signals, pipes, redirection and memory discipline.
- Deliver a responsive web interface, automated tests, containerized deployment and a live academic demonstration.

1.4 Project Scope and Workstreams

Table 1. Interdisciplinary workstreams and responsibilities

Workstream	Primary responsibility	Implemented outcome
DBE-DSD	Databases, backend APIs, security and frontend	Hybrid persistence, Spring Boot REST API, React application and deployment
DSA-3	Algorithms and complexity analysis	TextHack engine with 20 algorithms, demonstrations and complexity registry
OSSP	Operating-systems and systems-programming concepts	ShellForge v9 with processes, signals, pipes, redirection and diagnostics
Machine Learning	Embeddings, retrieval evaluation, classification and clustering	MiniLM pipeline, FiQA evaluation and ML insights page

Project contributor: SathyaMaragani. The repository records the four workstreams above as the project organization and integrates their outputs where appropriate. TextHack and the ONNX embedding path run inside the web backend; ShellForge remains a standalone systems-programming deliverable in the same repository.

2. SYSTEM DESIGN AND METHODOLOGY

2.1 Overall Architecture

The user interacts with a React single-page application. Requests pass through a Spring Boot API that authenticates the caller, applies role and document-level access rules, coordinates search or ingestion and composes responses from the data stores. The design avoids treating one database as a universal solution: each store owns the data shape and access pattern it handles best.

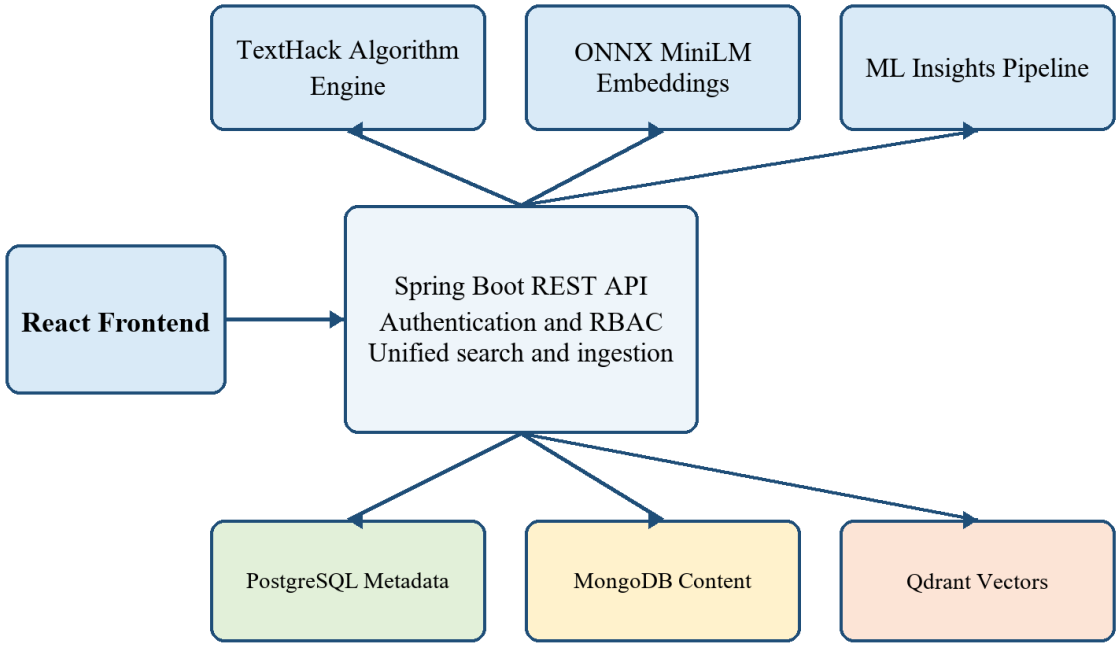


Figure 1. Logical architecture of the integrated platform

Table 2. Technology stack

Layer	Technology	Purpose
Frontend	React 19, React Router, Vite 8	Responsive authenticated single-page application
Backend	Java 21, Spring Boot 3.4.2	REST API, security, orchestration and health endpoints
Relational data	PostgreSQL 16	Users, roles, permissions, metadata, versions, grants and history
Document data	MongoDB 7	Extracted content, chunks, metadata and processing records
Vector data	Qdrant	384-dimensional cosine-similarity vectors and search filters
Embedding	ONNX Runtime and DJL tokenizer	In-process MiniLM query and chunk encoding
Algorithms	Plain Java TextHack	Exact, fuzzy, alignment, flow, approximation and randomized methods
Systems	C and POSIX APIs	ShellForge process, IPC, signal and redirection features
Delivery	Docker, GitHub Actions, Vercel and Render	Repeatable testing and cloud deployment

2.2 Hybrid Database Responsibility Model

Table 3. Storage responsibilities

Store	Authoritative data	Key identifiers and indexes
PostgreSQL	Users, roles, permissions, categories, documents, tags, versions, grants and search history	documents.id links every store; indexed ownership, category, status and user relationships

Store	Authoritative data	Key identifiers and indexes
MongoDB	Raw extracted text, word/character counts, source details, chunks, processing metadata and version snapshot	Unique postgres_document_id; title, department, keywords, status, version and timestamp indexes
Qdrant	One normalized 384-D vector per chunk with denormalized filter payload	postgres_document_id, category, department, language and processing_status payload indexes

Cross-store identity is explicit. PostgreSQL document identifiers are stored in MongoDB as postgres_document_id and in every Qdrant payload. MongoDB chunk identifiers such as doc7-chunk3 are repeated in Qdrant, allowing a semantic hit to be joined back to the readable chunk and relational metadata.

2.3 Document Ingestion Method

An upload is validated before any persistent record is created. The file name selects a format-specific extractor. UTF-8 decoding handles TXT and Markdown, Apache PDFBox extracts text layers from PDF files, and a ZIP plus StAX reader extracts WordprocessingML text from DOCX files. External entities and DTD processing are disabled for DOCX, decompressed XML is bounded, password-protected or corrupt PDFs are rejected, and image-only PDFs receive a clear no-text error because OCR is outside the current scope.

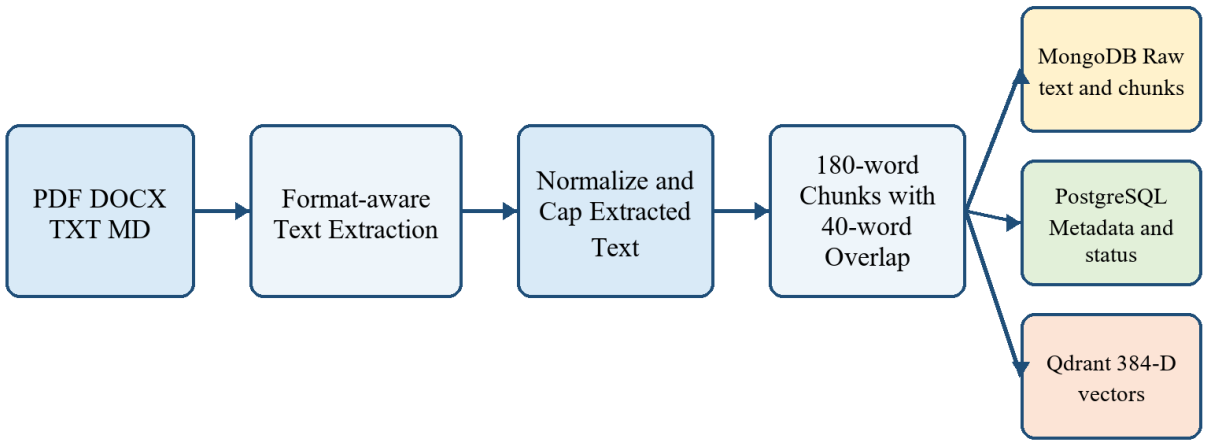


Figure 2. Document ingestion, chunking and indexing workflow

Files may be at most 10 MB and extracted text may be at most 1 MB. After line-ending normalization and blank-content checks, text is divided into 180-word windows with a 40-word overlap. PostgreSQL first records the document in PROCESSING state; MongoDB receives the content and chunks; PostgreSQL receives version 1; and Qdrant receives embeddings when the model and vector store are available. The final state is INDEXED when vectors are stored or UPLOADED when content is safe but semantic indexing is temporarily unavailable. Compensating cleanup removes earlier writes if a later non-vector step fails.

2.4 Search Methodology

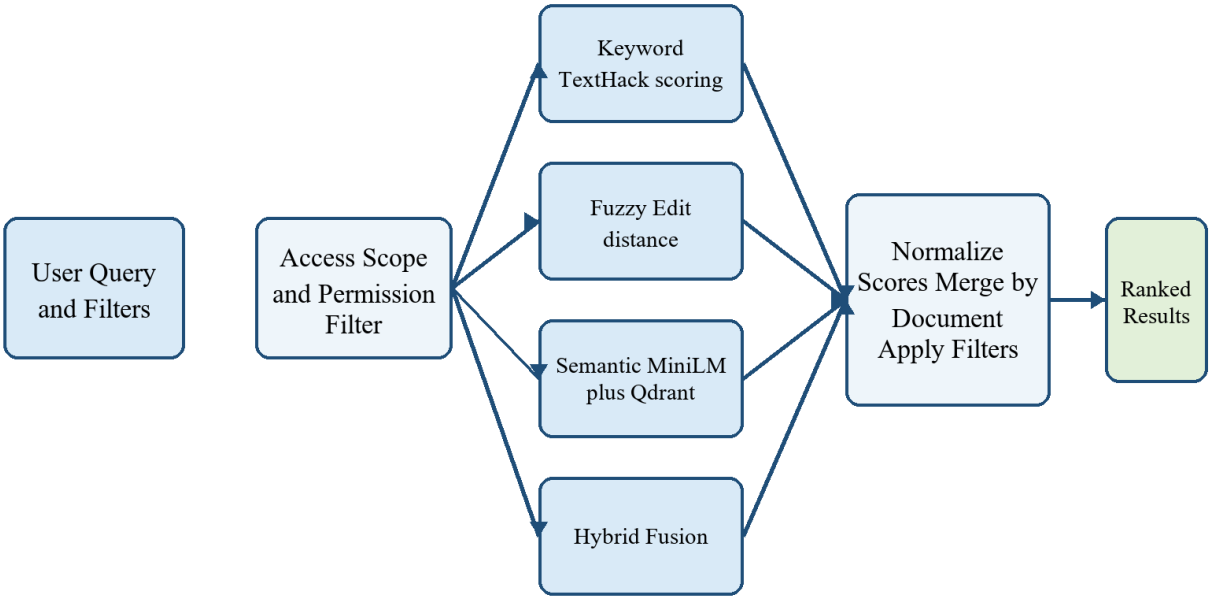


Figure 3. Unified search modes, permission filtering and ranking

Table 4. Search modes

Mode	Method	Best use
Keyword	TextHack exact and multi-pattern scoring over metadata	Known terms, names and phrases
Fuzzy	Damerau-Levenshtein-aware lexical scoring	Typos, transpositions and near matches
Semantic	MiniLM query embedding and Qdrant cosine search	Conceptual similarity and paraphrases
Hybrid	Normalized fusion of lexical and vector evidence	Balanced general search

Every mode applies the same access model. Administrators can see all documents; other users see documents they own or documents explicitly shared with them. Category and status filters are applied consistently. Search activity is recorded without allowing logging failures to break a successful search response.

2.5 Security and Access Model

Authentication uses signed JSON Web Tokens. Passwords are stored as BCrypt hashes. The API exposes a minimal public surface: login and health checks; document, search, administration and sharing operations require authentication and method-level authorities. Seed roles grant administrators full management, managers document creation and employees read access within ownership or explicit grants. Disabled accounts are rejected even when a previously issued token exists.

- Administrative APIs create users, change roles, enable or disable accounts and reset passwords.
- Document grants support listing, creation and revocation of READ access.
- Validation and global exception handling return stable error structures without leaking stack traces.
- Production health output hides internal component details while dedicated tests prevent accidental exposure.
- The frontend derives visible actions from the authenticated profile, but backend authorization remains authoritative.

3. IMPLEMENTATION

3.1 Backend and API

The backend follows controller, service and repository layers. Controllers map HTTP requests and responses; services own orchestration, authorization and business rules; repositories isolate PostgreSQL, MongoDB and Qdrant access. The

API covers authentication and profiles, health, document listing and pagination, upload and deletion, categories, document grants, administration, vector search, unified search, search history and TextHack demonstrations.

Table 5. Representative REST endpoints

Area	Endpoint examples	Behavior
Authentication	POST /api/auth/login; GET /api/auth/me	Issue token and return the current profile
Documents	GET/POST /api/documents; GET/DELETE /api/documents/{id}	List, upload, view and remove authorized documents
Search	POST /api/search; GET /api/search/history	Run four search modes and retrieve recent activity
Sharing	/api/documents/{id}/permissions	List, grant and revoke document access
Administration	/api/admin/users and /api/admin/roles	Manage users, roles, status and passwords
TextHack	/api/texthack/*	Pattern, similarity, citation and complexity demonstrations

3.2 Frontend Experience

The React application provides sign-in, a role-aware dashboard, global search, repository browsing, document viewing, upload, sharing, administration, TextHack demonstrations and ML insights. Search mode is represented in the URL so results remain reproducible. The dashboard presents recent documents and activity, while the repository offers pagination and filters. The document page exposes source, content, chunk and processing details when the caller is authorized.

The upload page mirrors server-side extension and size checks, accepts drag-and-drop entry from the dashboard, and explains that scanned PDFs require OCR. A server-gate component handles Render cold starts by polling health before allowing normal requests. Accessibility-oriented labels, responsive layouts down to mobile width, consistent badges and explicit empty/error states are covered by frontend tests.

3.3 TextHack Algorithm Engine

TextHack is dependency-free Java compiled directly into the Spring backend as a second source root. Core algorithms avoid java.util data structures and are accompanied by explicit time/space analysis. The engine supports exact pattern search, multi-pattern search, fuzzy similarity, sequence alignment, graph flow, approximation and randomized demonstrations.

Table 6. Implemented TextHack algorithms

Category	Algorithms	Key complexity or guarantee
String matching	Naive, KMP, Z, Rabin-Karp, Aho-Corasick	KMP/Z O(n); Aho-Corasick O(n + matches)
Suffix structures	Prefix-doubling suffix array, Kasai LCP	O(n log n) and O(n)
Dynamic programming	Levenshtein, two Damerau variants, Needleman-Wunsch, Smith-Waterman	O(nm); Levenshtein uses O(min(n,m)) space
Network flow	Ford-Fulkerson, Edmonds-Karp, Dinic, bipartite matching	Multiple strategies and graph guarantees
Approximation	Vertex cover, list scheduling, LPT	2-approximation and scheduling bounds
Randomized	Miller-Rabin, universal hashing, reservoir sampling, SplitMix64	Exact 64-bit primality witness set; one-pass sampling

The six TextHack suites contain 452 assertions with zero failures and zero compiler warnings. Randomized cross-validation generates 4,000 string-matching cases and requires Naive, KMP, Z and Rabin-Karp implementations to agree. Tests include overlapping matches, repetitive worst cases, Unicode, embedded NUL characters, Carmichael numbers and flow cases that require reverse edges.

3.4 Machine Learning Pipeline

Semantic retrieval uses sentence-transformers/all-MiniLM-L6-v2 through ONNX Runtime inside Java. The DJL tokenizer produces input identifiers, attention masks and token-type identifiers; the backend performs attention-mask-aware mean pooling and L2 normalization. This makes dot product equal cosine similarity in Qdrant. Java ONNX output was numerically matched against Python, with approximately 1.5e-7 maximum element difference and identical top-five retrieval across 32 regression queries over 705 vectors.

The ML work separates deterministic database fixtures from the retrieval corpus. FiQA-2018 supplies 57,638 documents, 648 test queries and 1,706 relevance judgments. A bounded, reproducible subset of 150 queries and 7,087 chunks compares TF-IDF, MiniLM and BGE-small. Dense models significantly outperform TF-IDF; MiniLM is selected provisionally because it is about 2.5 times faster and its difference from BGE is not statistically significant on the bounded subset.

Table 7. Retrieval evaluation on the bounded FiQA subset

Model	nDCG@10	MRR	Recall@10
TF-IDF	0.3709	0.4182	0.4617
MiniLM-L6	0.6240	0.7120	0.6739
BGE-small	0.6533	0.7322	0.7089

3.5 Classification, Clustering and Insights

The demo corpus contains 315 policy documents: 21 topics, six organizational categories and 15 near-identical versions per topic. Random document splits would leak topic copies into both training and validation, so evaluation groups by topic. Character TF-IDF with multinomial Naive Bayes is selected after leave-one-topic-out validation. It scores 0.83 accuracy and 0.78 macro-F1 on six locked unseen topics, but 0.57 accuracy across all 21 leave-one-topic-out folds. The difference is reported rather than hidden because the independent sample is small and category labels are organizational rather than purely topical.

Unsupervised analysis compares k-means variants, agglomerative clustering, mini-batch k-means and DBSCAN. DBSCAN automatically finds 21 topic clusters with topic ARI 0.98 and 11 noise documents. At six clusters, k-means on a 17-component PCA representation achieves category ARI 0.60. The frontend Insights page displays the generated results, including category performance, cluster structure and t-SNE visualization data.

3.6 ShellForge Systems Programming

ShellForge is a Unix-like shell implemented in C as the OSSP contribution. Version 9 implements a dynamic REPL and parser; built-ins such as cd, pwd, env, clear, help and exit; fork, execvp and waitpid execution; SIGINT, SIGCHLD and SIGTSTP handling; two-process pipelines; and input, output, append and error redirection. File descriptors are closed deliberately to prevent deadlocks, child signals are restored to defaults, and SIGCHLD is blocked around foreground waits so exit status is not lost.

The project provides AddressSanitizer, Valgrind and GDB workflows and checks every allocation and system call. Six weekly suites total 148 passing assertions. It builds with GCC 13 in a Linux container on Windows and requires no change to build on a native Linux host.

4. EXPERIMENTS AND VALIDATION

4.1 Automated Test Strategy

Validation combines pure unit tests, database-backed integration tests, randomized algorithm checks, model comparisons, production-profile checks, frontend component tests and deployed-stack smoke tests. Integration tests use seeded PostgreSQL, MongoDB and Qdrant services and remove temporary uploads so fixture state remains stable.

Table 8. Verification summary

Subsystem	Evidence	Result
Backend	Unit plus PostgreSQL/MongoDB/Qdrant integration and demo corpus	191 tests passing
Frontend	Vitest component and behavior suite	148 tests passing; production build succeeds
TextHack	Six algorithm suites and randomized cross-validation	452 assertions passing; zero lint warnings
ShellForge	Weeks 4-9 shell suites	148 assertions passing
Machine learning	Pipeline, embedding structure/behavior and insight consistency	25 offline checks plus two model-dependent checks
Deployment	Nginx/Vercel path, auth, search, TextHack and resource tests	28 of 28 smoke checks passing

4.2 PDF and DOCX Validation

The document extractor tests build real PDF files and DOCX ZIP packages in memory. They verify multi-page PDF extraction, Word paragraphs, tabs and line breaks, Unicode, corrupt files, password protection, image-only PDFs, extension mapping and the extracted-text cap. A crafted external-entity DOCX confirms that the XML parser never reads local or remote entity content. API integration tests then upload both formats, verify document type and MIME metadata, inspect stored text and chunks and clean up every created record.

4.3 Retrieval and Model Evaluation

Retrieval quality uses Recall at K, mean reciprocal rank and normalized discounted cumulative gain. Paired bootstrap testing shows both dense models beat TF-IDF with $p < 0.0001$, while BGE-small versus MiniLM is not significant at $p = 0.1383$. Results on the bounded subset are intentionally labeled as inflated relative to the full corpus and are not presented as published BEIR scores.

Classification evaluation locks one topic per category before tuning and repeats leave-one-topic-out evaluation across all 21 topics. Clustering is evaluated against both category labels and topic labels using adjusted Rand index, normalized mutual information, homogeneity, completeness and silhouette. This protocol exposes the difference between memorizing near-duplicates and generalizing to unseen topics.

4.4 Resource and Deployment Testing

Under Render's 512 MB and 0.1 CPU free-tier cap, memory rose from 391 MiB after startup to 446 MiB after repeated searches, 16 concurrent semantic searches and a 950 KB upload, without an out-of-memory kill. Timings were 0.2-0.4 seconds for keyword, 0.4-0.9 seconds for semantic and 0.5-0.6 seconds for hybrid search; cold starts are slower while the JVM and model reload.

5. RESULTS AND DISCUSSION

5.1 Completed Capabilities

- A live authenticated platform with dashboard, search, repository, document viewer, upload, sharing, administration, TextHack and ML insights pages.
- Hybrid persistence across PostgreSQL, MongoDB and Qdrant with stable cross-store identifiers and compensating cleanup.
- Four search modes that combine transparent lexical algorithms with semantic retrieval while enforcing one access policy.
- In-process MiniLM embeddings with Python/Java numerical equivalence and no separate model microservice.
- Secure PDF and DOCX text extraction in addition to UTF-8 text and Markdown uploads.
- A comprehensive first-principles algorithm library and a standalone systems-programming shell.
- Leakage-aware ML evaluation that reports both strong results and uncertainty instead of relying on a misleading random split.
- Automated CI gates and a resource-constrained cloud deployment verified through the public frontend and backend.

5.2 Quantitative Outcomes

Table 9. Selected project outcomes

Measure	Outcome	Interpretation
TextHack coverage	20 algorithms; 452 assertions	Broad algorithmic scope with explicit correctness and complexity evidence
Semantic retrieval	MiniLM nDCG@10 0.6240	Substantial improvement over TF-IDF 0.3709 on bounded FiQA subset
Classification	0.83 locked-test accuracy; 0.57 across 21 topics	Promising but sensitive to which independent topics are held out
Unsupervised topics	DBSCAN 21 clusters; topic ARI 0.98	Embeddings separate the synthetic policy topics strongly
Backend quality	191 tests	Business logic, security, databases, search and deployment profile covered
Deployment footprint	Approx. 446 MiB under stress	Fits the 512 MB Render free tier with limited margin

5.3 Limitations

- Scanned or image-only PDFs require OCR, which is not implemented. Password-protected PDFs are rejected.
- The 10 MB file cap and 1 MB extracted-text cap bound memory and indexing cost but exclude very large documents.
- Embedding runs synchronously in the upload request; large documents can exceed a free proxy's request timeout even if backend processing later completes.
- The deployment relies on free services that sleep, suspend or delete idle resources. Cold starts can exceed 100 seconds.
- The 315-document insights corpus is synthetic and contains near-duplicate versions; its results demonstrate method correctness, not production accuracy on a real enterprise corpus.
- The bounded FiQA comparison is reproducible and statistically tested but does not replace full-corpus benchmarking.
- ShellForge is a standalone OSSP deliverable and is not exposed as a production web shell, which avoids a serious remote-command security risk.
- The Indian-language Wikipedia corpus originally considered for TextHack remains outside scope because acquisition, licensing and storage require a separate data decision.

6. DEPLOYMENT AND OPERATIONS

6.1 Cloud Topology

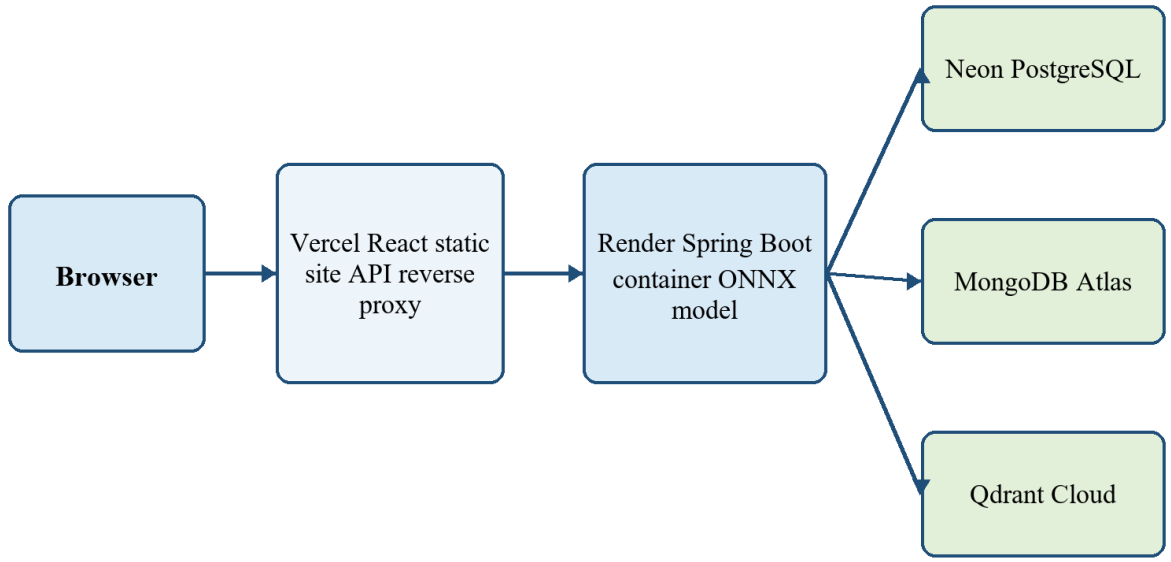


Figure 4. Cloud deployment topology

Table 10. Production services

Component	Service	Operational role
Frontend	Vercel Hobby	Builds the React application and proxies /api to the backend
Backend	Render free web service	Runs the Spring Boot container, TextHack and ONNX MiniLM
Relational database	Neon PostgreSQL	Pooled SQL connection for identity and metadata
Document database	MongoDB Atlas	Managed storage for content and chunks
Vector database	Qdrant Cloud	Managed cosine search with gRPC/TLS access
Continuous integration	GitHub Actions	Runs frontend and backend quality gates before deployment

6.2 Production Safeguards

- The backend Docker image pins and checksum-verifies the tokenizer and ONNX model.
- The production JVM uses a constrained heap, C1-only compilation, two malloc arenas and a class-data-sharing archive to fit the memory budget.
- Render exposes /api/health and limits request threads and database connections.
- Vercel rejects a missing or malformed backend origin at build time and disables caching for per-user API responses.
- The frontend waits for the sleeping backend to become healthy instead of immediately presenting an authentication failure.
- The first administrator is created from temporary environment variables only when no administrator exists; the variables are removed after bootstrap.

6.3 Live Availability

The production user interface is available at https://ekipsearch.vercel.app. The public backend health endpoint is available at https://enterprise-knowledge-intelligence-0hg2.onrender.com/api/health. The PDF and DOCX release was deployed from commit 69e5e43 after both frontend and backend GitHub workflows passed.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The project delivers a working enterprise knowledge platform rather than a collection of disconnected subject exercises. Its design assigns metadata, content and vectors to suitable data stores; exposes them through a secured service; combines exact, fuzzy and semantic evidence in search; and presents the result through a responsive web interface. TextHack, ShellForge and the ML experiments provide independent algorithmic, systems and analytical depth, while the deployed application proves that the major parts can operate together under strict resource limits.

The strongest result is not a single metric. It is the combination of explainable algorithms, reproducible evaluation, defensive ingestion, permission-aware retrieval, automated testing and live deployment. The documented limitations also identify where academic demonstration ends and production hardening would begin.

7.2 Future Work

- Add OCR for scanned PDFs with explicit language selection and confidence reporting.
- Move large-document embedding to a background job and expose retry/progress state.
- Evaluate quantized MiniLM if future memory pressure outweighs the cost of rebuilding vectors.
- Evaluate category suggestions on real organizational documents before enabling automated filing.
- Run full-corpus FiQA evaluation and extend retrieval tests to multilingual and domain-specific corpora.
- Add document re-indexing, vector rebuild and retention workflows for model upgrades.
- Add audit reporting for grants, administrative changes and sensitive document access.
- Extend ShellForge with job control, process groups, FIFOs and virtual-memory demonstrations while keeping it isolated from the public web application.

REFERENCES

[1] Enterprise Knowledge Intelligence Platform repository and project documentation, GitHub, 2026.

[2] N. Thakur, N. Reimers, A. Rucklé, A. Srivastava and I. Gurevych, 'BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models,' NeurIPS Datasets and Benchmarks, 2021.

[3] N. Reimers and I. Gurevych, 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,' EMNLP-IJCNLP, 2019.

[4] Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/.

[5] PostgreSQL Documentation, https://www.postgresql.org/docs/.

[6] MongoDB Manual, https://www.mongodb.com/docs/.

[7] Qdrant Documentation, https://qdrant.tech/documentation/.

[8] Apache PDFBox Documentation, https://pdfbox.apache.org/.

[9] ONNX Runtime Documentation, https://onnxruntime.ai/docs/.

[10] Project records: backend API, testing, ML evaluation, TextHack complexity, ShellForge test and cloud deployment documentation in the repository.